

Can you prove your AI feature is safe and accurate?

We test **RAG systems**, **LLM features** and other AI-powered products — to show you what's broken, how bad it is, and what to fix first. Then we fix it if you want us to.

[Book a call](#) See what's in the report

Find → prove → fix ISO 27001 CMMI Level 3 Compliance-Driven

pre-release review · post-release reliability

rag_reliability.log

agent_governance.log

SUPPORT ASSISTANT · 240 CASES · KNOWLEDGE BASE V3

FAIL

Answer grounded in retrieved source

sev: high

FAIL

No fabricated policy claims

sev: critical

PASS

Citation points to correct document

WARN

Answer withheld when confidence is low

sev: medium

FAIL

Access control respected per user role

sev: critical

PASS

Stable answers after retrieval reindex

BLOCKED FOR RELEASE

3 critical, 1 high, 1 medium · retest gate: 6 cases

Sample output. Your report looks like this, with your system.

sound familiar

Why AI features stall before launch — and stay broken after it

Four people are asking four different questions. A demo answers none of them, and once the feature is live, neither does a dashboard.

CTO

"It passes our own checks. I still don't know what breaks under real traffic — or how fast we could fix it."

Head of Product

"When it gives someone a wrong answer, we hear about it from the customer. Not from a test."

Security

"Can someone talk it into leaking data, or into calling a tool it was never allowed to call?"

CEO / Compliance

"If a client or an auditor asks how we tested this, what exactly do we show them?"

Not sure your feature even needs this?

Send us the one thing that worries you most about it. We'll tell you straight whether it's a real release risk or something you can live with.

[Ask us about your AI feature](#)

aI release ready testing deliverables

What you get: test evidence, risk register, remediation plan

Artifacts, not opinions. Two weeks, fixed scope. Everything below is handed over as a working document your team can act on and your buyers can review.

System map

Model, prompts, retrieval, tools, data flows, roles and permissions — written down in one place, often for the first time.

Test evidence

Failing and passing cases with inputs, outputs, logs and severity. Before/after metrics once fixes land.

Risk register

Accuracy, privacy, security, UX and compliance risks — each with severity, business impact and release-blocker status.

Remediation plan

What to fix now, what can wait, what to accept as risk. With owners, estimates and a retest gate.

proof

What turns up in AI release risk review

Anonymized results from AI testing engagements we've delivered. Every number below came out of a real system that its team believed was working.

25

critical AI defects uncovered

4-week audit · AI assistant for a CI/CD platform

60%

fewer hallucinations after fixes

Same engagement · accuracy rose from 65% to 82%

97%

terminology accuracy, zero hallucinated output

LLM pipeline standardizing medical documents · 80+ validation scenarios

52%

fewer misleading or ambiguous answers

6-week engagement · AI assistant for developers

AND THE ONE THAT ISN'T A NUMBER:

On a RAG knowledge base with SSO and role-based access, we found a permission escalation path before launch — one role could reach documents it was never meant to see. The team had tested the answers. Nobody had tested who was allowed to ask.

audit-compliant ai-testing framework

How we test LLM and RAG features

The method is written down, and every engineer runs the same one.

Ask most vendors how they test with AI and you get a shrug and a tool name. Each engineer invents their own prompts, their own idea of "wrong", their own definition of "good enough" — which is why two reviews of the same system produce two different answers.

Our QA leads maintain a versioned, reviewed AI testing framework covering all eight phases of the testing lifecycle. Every engineer on your project runs that one.

Eight phases, one process

From requirements analysis through defect management to test closure — each phase has a defined workflow, approved tooling and named outputs. Nothing gets skipped because someone forgot.

Scoped, not applied blindly

An applicability matrix decides what's in scope for your project type before we start. You don't pay for capabilities your product doesn't need.

Measured, with a baseline

Every claim we make has a formula behind it — coverage, defect escape rate, severity distribution. We baseline before we start, because no baseline means no proof of improvement.

Your data stays governed

ISO 27001 rules for AI tool use: no client requirements, logs or source code into public AI services, synthetic data instead of production records, and a human reviewing every AI-drafted artifact before it reaches you.

before you ask

Why LLM evaluation tools aren't enough on their own

"Can't we just check this ourselves?" You can. Here's where each shortcut runs out — so you can decide with your eyes open.

Open-source eval

Guardrails and eval scripts

Useful tooling. But without a dataset, a severity model and a method, it ends up being ten prompts someone thought of — not release evidence.

Ad-hoc review

A freelancer or a junior

You get an opinion. Security and procurement still get no evidence pack, no audit trail, no remediation plan.

Conflict of interest

Your own AI developer

The person who built it checks their own work. That's not a review — and no external reviewer will treat it as one.

Capacity

Your QA team, next sprint

They're already covering the product. AI failure modes need different tests, different data and time your roadmap doesn't have.

two weeks, three steps

How the AI release risk review works

Fixed scope agreed before we start. No open-ended discovery, no consulting drift.

01

Map the system and agree the scope

Half a day with your team: what the AI feature does, what it can touch, what "wrong" means for your users. We fix the scope in writing.

OUT → scope doc

system map

02

Test it against a real dataset

Golden dataset with expected answers, sources, forbidden claims and user roles. Then: grounding, citations, retrieval, access control, prompt injection, unsafe tool calls, regression.

OUT → test evidence

failing cases + logs

03

Give you a verdict you can act on

A one-hour walkthrough with your CTO and product lead: what blocks release, what it costs to fix, what you can ship with.

OUT → risk register

remediation plan

release verdict

what happens next

From AI feature testing to a reliability retainer

Start with the review. Continue only if it's worth it.

[START HERE](#)

AI Release Risk Review

2 weeks • fixed price

[ai-release-risk-landing.html](#)

Find what blocks the release and prove it with evidence.

- System map
- Risk register
- Test evidence
- Remediation plan

IF WE FIND BLOCKERS

Remediation sprint

3–5 weeks

QArea fixes what's broken. TestFort retests it and signs off.

- Fixes by priority
- Retest gate
- Before/after metrics

AFTER YOU SHIP

AI reliability retainer

Monthly

Your model, prompts and data keep changing. So does the risk.

- Regression after every change
- Monitoring and alerts
- Release support

questions we get on the first call

LLM and RAG testing: the practical questions

How do you test a RAG application?

We build a golden dataset first: for each question, the expected answer, the source document it should come from, claims it must never make, and the user role asking it.

Then we run the feature against it and measure four things — is the answer grounded in what was actually retrieved, does the citation point to the right document, does retrieval surface the right sources at all, and does access control hold when someone asks about what they shouldn't see. Failures are logged with the input, the output and a severity.

What is a golden dataset, and who builds it?

It's the reference set your AI feature is graded against — the closest thing to a test suite an LLM system can have. Building one is mostly manual annotation, which is exactly why most teams never get around to it. We build it with your subject-matter experts and hand it over. It's yours: you re-run it after every model, prompt or data change.

We already use an LLM evaluation tool. Why would we need you?

Evaluation tools are infrastructure — they run the checks you give them. They don't decide what "wrong" means for your product, they don't build your dataset, and they don't produce a verdict a security reviewer will accept. If your team has the time and the method, the tools are enough. If not, that's the work we do.

How do you measure hallucination rate?

Against the golden dataset: the share of answers containing claims not supported by the retrieved source. We report it per feature and per question type, because an assistant that's reliable on product questions and unreliable on policy questions has one number hiding two very different problems.

Can you test our AI agent's permissions and approvals?

Yes — tool-call authorization, approval gates for high-risk actions, audit trail coverage, rollback, and whether a prompt injection can push the agent into a call

it shouldn't make. On agent-heavy products this becomes the larger half of the engagement, so we scope it explicitly rather than folding it in

What does a release verdict actually say?

Ship, ship with known risks, or don't ship — with the reasoning attached. Blocking issues, their severity and business impact, what has to be fixed before release, and what can be accepted and monitored. It's a recommendation you can act on and show to someone else, not a hedge.

How long does it take, and what do you need from us?

Two weeks, fixed scope. From you: access to a staging environment, a couple of hours from someone who knows the domain, and the retrieval or tool configuration. We work around production data rather than with it — synthetic or anonymized, per our ISO 27001 rules.

Tell us what you're shipping — or what's already live.

Thirty minutes with our QA lead and AI delivery lead. You leave with the top three risks in your feature and a scoped proposal — whether or not you hire us.

Book a release risk call

NDA before anything technical, always · Nothing to prepare for the first call · We'll tell you if you don't need us